

미션 16 — 모델 포맷 변환 실습 요약 보고서

4팀 신호정 · 2026-07-15

1. 개요

미션 06(흉부 X-ray 폐렴 진단)의 MobileNetV3-Large 전이학습 모델을 재학습한 뒤 3종 포맷으로 추출하고, ONNX 기반 추론 코드로 동작을 검증하며 포맷 간 성능을 비교했다.

산출 포맷	파일명	방식	크기
PyTorch FP32	mission_16_mobilenetv3.pth	state_dict 저장	16.24 MB
PyTorch INT8	mission_16_mobilenetv3_quant.pth	PTQ(정적 양자화) 후 TorchScript 저장	4.47 MB
ONNX	mission_16_mobilenetv3.onnx	opset 17, Fixed Shape (1, 3, 224, 224)	16.03 MB

평가 원칙(미션 06 승계): 의료 도메인이므로 accuracy 단독으로 판단하지 않는다. Recall(폐렴)과 FN(위음성) 개수를 우선하며, 양자화 전후 FN 변화가 본 보고서의 핵심 논점이다.

제출 노트북: modeling.ipynb (학습 + 3종 추출), inference.ipynb (ONNX 추론 + 3종 비교).

2. 데이터 및 기준 모델

- 데이터:** Kaggle Chest X-Ray Pneumonia. 원본 val이 16장뿐이라 미션 06과 동일하게 train+val 병합 후 stratified 80:20 재분배(train 4,185 / val 1,047), test 624장은 원본 유지. 재분배 결과는 미션 06 기록과 클래스별 매수까지 완전히 일치함을 검증했다.
- 전처리(미션 06 승계):** 그레이스케일 → CLAHE(clip 2.0, tile 8×8) → 가로세로비 유지 패딩 → 224×224 → 3채널 → ImageNet 정규화. 침윤·경화 소견 보존을 위해 대비(contrast) 증강은 금지.
- 학습:** 양자화 대응 변형 `quantization.mobilenet_v3_large(quantize=False)` (E 섹션 PTQ에서 `fuse_model()` 재사용 목적), AdamW(lr 1e-4), 클래스 가중치 CrossEntropy, BF16 AMP, 최대 15 epoch + 조기 종료(patience 3), SEED 42.
- 기준 모델 선택:** (Recall, accuracy) 사전식 비교로 epoch 3 확정 — val accuracy 0.9522, **Recall(폐렴) 1.0000, FN 0 / 777**.

주목할 실험 결과: epoch 6은 val accuracy 0.9857로 전체 최고였으나 **FN 8건**(Recall 0.9897)이 발생해 배제되었다. 일반적인 기준(val loss·accuracy)이라면 epoch 6이 선택되는 상황으로, "accuracy가 올라도 FN이 늘면 개악"이라는 미션 06 원칙이 실제로 작동한 사례다. 이 모델·데이터에서 특이도 상승과 Recall 유지는 맞교환 관계임이 확인되었다.

3. 3종 포맷 추출 및 검증

단계	방법	검증 결과
FP32	<code>model.eval()</code> 후 <code>state_dict</code> 저장	재로드 출력 최대 오차 0.00 (<code>torch.allclose</code> 통과)
INT8	eager mode PTQ: fuse → <code>fbgemm</code> qconfig → prepare → 보정 (train 300장, 클래스별 150장 균형) → convert → <code>torch.jit.script</code>	재로드 추론 정상, 3.64배 압축
ONNX	레거시 <code>exporter(dynamo=False)</code> , opset 17, Fixed Shape	<code>onnx.checker</code> 통과, PyTorch 대비 최 대 오차 1.43e-06 (<code>atol=1e-4</code> 통과)

- ONNX는 요구 조건이 opset 17이므로 레거시 TorchScript 기반 exporter를 명시 사용했다 (torch 2.9+ 기본인 dynamo exporter는 opset 18 이상 대상).
- `torch.ao.quantization`·`torch.jit.script`는 torch 2.10에서 deprecated 경고를 출력하나 동작은 정상임을 사전 검증했다. 향후 동일 파이프라인은 torchao(pt2e) API로의 이행이 필요하다.

4. 3종 포맷 비교 평가 (test 624장)

전처리를 numpy-cv2만으로 재구현해 torchvision 파이프라인과 최대 오차 0.00 일치를 확인한 뒤, 3종 모델에 동일한 입력을 사용했다. 지연시간은 단일 스레드·단건 입력·warmup 10회 후 50회 평균으로 측정했다(온디바이스 단건 추론 시나리오, x86 CPU).

포맷	용량(MB)	accuracy	Recall(폐렴)	FN	지연시간(ms)
FP32	16.24	0.7051	1.0000	0	20.20 ± 4.23
INT8	4.47	0.6891	1.0000	0	11.50 ± 1.97
ONNX	16.03	0.7051	1.0000	0	9.40 ± 1.21

4-1. FN 분석 (핵심 논점)

- FP32 vs ONNX**: 예측이 624/624 완전 일치(logits 수준까지 동일). ONNX 변환은 수치 등가이며 FN 변화가 없다. ONNX의 가치는 압축이 아니라 런타임 이식성과 속도(2.15배)다.
- FP32 vs INT8**: 예측 26/624건이 변화했으나 전부 FP(정상→폐렴) 방향이다. accuracy는 1.60%p 하락했지만 Recall(폐렴) 1.0, FN 0이 그대로 유지되었다. 위음성 최소화가 최우선인 의료 도메인에서 안전한 방향의 열화이며, 보정 데이터 확대나 QAT 재검토가 필요한 수준(FN 증가)에 해당하지 않는다.
- 속도의 역전**: FP32 그래프의 ONNX가 INT8보다 빨랐다. onnxruntime의 그래프 최적화 (연산 융합·메모리 플래닝)가 효과적인 반면, eager PTQ 산출물은 TorchScript 인터프리터 오버헤드를 안기 때문으로 추정된다. INT8의 확실한 이점은 용량이며, 속도 이득은 런타임 최적화 수준에 좌우된다.

4-2. val 대비 test 성능 하락에 대하여

val accuracy 95.2% 대비 test accuracy 70.5%는 이 데이터셋의 알려진 분포 차이다. 미션 06에서도 Optuna 튜닝을 거친 4개 모델 전부 test 특이도가 45~54%에 그쳤다 (MobileNetV3 82.5%, EfficientNet-B0 81.3%). 연장 학습으로 특이도를 올리는 시도는 Recall 하락(FN 8)과 맞교환되어 배제했다(2절). 우선 지표인 Recall·FN은 미션 06 최적 모델 (EfficientNet-B0: Recall 1.0, FN 0)과 동일 수준으로, 포맷 변환 전후 비교라는 본 미션 목적에 충분한 기준선이다.

5. On-Device 배포 관점 결론

- **메모리·저장소**: INT8 4.47 MB는 모바일 앱 번들·RAM 상주에 부담 없는 수준. FP32/ONNX 16 MB도 배포 가능하나 4배 차이는 저사양 기기·다중 모델 구성에서 유의미하다.
- **지연시간**: 단일 스레드 최악 조건에서 9~20 ms로 단건 판독 보조 시나리오에 충분하다. 단, 본 측정은 랩톱 x86 기준이므로 모바일 AP에서는 2~5배 여유를 잡아야 한다(추정).
- **백엔드 이식성**: 본 INT8은 x86 `fbgemm` 기준이다. ARM 모바일 배포 시 `qnnpack` (PyTorch/ExecuTorch) 계열로 재양자화·재검증이 필요하다. ONNX 경로는 ORT Mobile, NNAPI/CoreML EP 등으로 확장할 수 있어 프레임워크 독립성이 가장 높다.
- **의료 원칙**: 이 보정 조건에서는 양자화 후에도 FN 0이 유지되어 INT8 배포가 타당하다. 보정 데이터·모델이 바뀌면 FN 재검증이 필수다.
- **종합**: 저장소·메모리 제약이 크면 INT8, 이식성·속도가 우선이면 ONNX. 두 경로 모두 기준(FP32) 대비 Recall(폐렴)·FN 열화가 없음을 확인했다.

6. 디버깅 요약 (상세: DEBUG_LOG.md)

#	증상	원인	해결
1	<code>import onnxruntime</code> 시 <code>libcudart.so.12</code> 로드 실패	GPU 빌드가 요구하는 CUDA 런타임이 torch 동봉본뿐인데 알파벳순 import로 onnxruntime 이 먼저 로드됨	torch → onnxruntime 순서로 조정
2	학습이 epoch 1에서 무한 정지 (GPU 유향)	CUDA 초기화 후 fork된 DataLoader 워커가 종료·재개 신호에 미응답 (WSL2+Jupyter fork 불안정)	<code>/proc</code> 스택 덤프로 교착 지점 특정 후 <code>num_workers=0</code> 고정
3	체크포인트 재로드 시 <code>UnpicklingError</code>	PyTorch 2.6+ <code>weights_only=True</code> 기본값이 metrics 내 numpy 스칼라를 거부	지표를 파이썬 기본 타입으로 변환 후 저장

7. 실행 환경 및 재현

- WSL2(Ubuntu) + RTX 5050 Laptop, Python 3.12.13, torch 2.10.0+cu128, torchvision 0.25.0, onnx 1.21.0, onnxruntime 1.27.1, opencv 4.13.0
- 난수 고정: SEED 42 (random / numpy / torch / torch.cuda)
- 실행 순서: `modeling.ipynb` → `inference.ipynb` (데이터 경로는 노트북 상단 `LOCAL_DATA_ROOT` 상수 참조, Colab 실행 시 kagglehub 자동 분기)